

Metodologia de reconocimiento

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El reconocimiento es cartografiar la superficie de ataque: que puertos estan abiertos, que servicios corren y en que version. Se hace en capas de lo amplio a lo profundo para no perder tiempo ni hacer ruido: primero todos los puertos sin scripts, despues version y scripts solo en los abiertos, y por ultimo enumeracion especifica de cada servicio.

Escanear versiones de los 65535 puertos es lentisimo e inutil. Separar 'descubrir puertos' de 'analizar servicios' hace el recon rapido y ordenado. Y cada servicio se enumera distinto, asi que primero necesitas saber cuales hay antes de elegir la tecnica.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Es SIEMPRE la primera fase de una room. Cada servicio que descubras deriva a su enumeracion propia (445 -> smb-enum, 80 -> web, 88/389 -> ad-modelo).

Cómo identificarla

- Un puerto abierto con una version concreta: tu primera pista para buscar CVEs.
- Hostnames en certificados TLS o en scripts de nmap: anadelos a /etc/hosts.
- Servicios en puertos no estandar (labs que los mueven a proposito).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara el numero de puertos abiertos entre el barrido rapido y el escaneo con -sV: deben coincidir.

Evidencia esperada: Una lista de puertos open, y en el segundo escaneo, version y banner de cada servicio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap/all_ports.txt
```

Objetivo y contexto: Barrido de amplitud: los 65535 puertos sin scripts es rapido. -Pn evita que nmap descarte el host por no responder al ping (comun en labs).

Resultado esperado: Solo la lista de puertos open. Copia esos numeros para el segundo escaneo; ignora versiones aqui.

Capa 2: prueba documentada

```
nmap -sC -sV -Pn -p <PUERTOS> $IP -oN nmap/services.txt
```

Objetivo y contexto: Profundidad solo donde hace falta: version exacta (-sV, base de los CVEs) y scripts por defecto (-sC) que sacan datos gratis.

Resultado esperado: Versiones, banners, hostnames en certificados y hallazgos de scripts (smb anonimo, titulos web). Cada version -> searchsploit.

Anatomía de los payloads

La primera prueba es `nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap/all_ports.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una lista de puertos open, y en el segundo escaneo, version y banner de cada servicio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Barrido de los 65535 puertos sin scripts, rapido, para saber donde mirar.
- Segundo escaneo con -sC -sV SOLO sobre los puertos abiertos, para versiones y datos gratis.
- Asocia cada puerto a su enumeracion (web, SMB, DNS, dominio) y profundiza.
- Anota versiones y hostnames; alimentan la busqueda de CVEs y el resto del ataque.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya tienes la lista completa de puertos/servicios de una fase anterior (no repitas el barrido).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

No puedes atacar lo que no sabes que existe: primero puertos (amplio y rapido), luego versiones, luego enum por servicio. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (recon-metodologia)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.